Universidad Israel

DEPARTAMENTO: Ciencias de la Ingeniería **PROFESOR:** Mgs. Pablo Recalde
CARRERA: Informática **ESTUDIANTE:** Stephany Carolina Aldás Pérez
CURSO: 8vo **PARALELO:** "B" **DESCRIPCIÓN:** Tarea – Semana 2
ASIGNATURA: Tendencias Tecnológicas **SEMESTRE:** 2026B

TEMA: Investigación de arquitecturas de redes neuronales: MLP, CNN, RNN y LSTM

1. Introducción

Una red neuronal artificial (RNA) es un modelo computacional inspirado en el cerebro que aprende patrones a partir de datos. Su unidad básica es la **neurona artificial**: recibe entradas, las multiplica por unos **pesos**, suma un **sesgo** y aplica una **función de activación** que decide su salida.

$$y = f(\Sigma\, w_i \cdot x_i + b)$$

Donde x son las entradas, w los pesos, b el sesgo y f la función de activación (sigmoide, tanh, ReLU o softmax).

Las neuronas se organizan en capas, y la forma en que se conectan define la **arquitectura** de la red. Cada arquitectura está diseñada para un tipo de dato: el MLP para datos tabulares, la CNN para imágenes, y la RNN y la LSTM para secuencias como texto, voz o series de tiempo.

Objetivo: Describir la estructura, el funcionamiento, las ventajas, las limitaciones y las aplicaciones de las arquitecturas MLP, CNN, RNN y LSTM, y compararlas entre sí.

2. MLP – Perceptrón Multicapa (Multilayer Perceptron)

Definición. El MLP es una red neuronal feedforward (de propagación hacia adelante) formada por varias capas de neuronas totalmente conectadas. La información fluye en una sola dirección, de la entrada a la salida, sin ciclos.

Historia. Parte del perceptrón simple de Frank Rosenblatt (1958), que solo resolvía problemas linealmente separables. Minsky y Papert (1969) demostraron que no podía resolver el problema XOR. En 1986, Rumelhart, Hinton y Williams popularizaron el algoritmo de retropropagación (backpropagation), que permitió entrenar redes con capas ocultas.

Arquitectura.

1. **Capa de entrada:** una neurona por cada característica del dato.
2. **Capas ocultas:** una o más; cada neurona se conecta con todas las de la capa anterior (capa densa o fully connected).
3. **Capa de salida:** una neurona por clase (clasificación, con softmax) o una sola para regresión.

Funcionamiento.

1. **Propagación hacia adelante:** cada capa calcula sus salidas y las pasa a la siguiente hasta producir una predicción.
2. **Cálculo del error:** una función de pérdida (error cuadrático medio o entropía cruzada) compara la predicción con el valor real.
3. **Retropropagación:** con la regla de la cadena se calcula cuánto contribuyó cada peso al error.
4. **Actualización de pesos:** el descenso del gradiente ajusta los pesos.

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Donde η es la tasa de aprendizaje y L la función de pérdida.

Ventajas.

- Es un aproximador universal: con suficientes neuronas puede aproximar cualquier función continua.
- Es sencillo de implementar y funciona bien con datos tabulares.

Limitaciones.

- Tiene muchos parámetros, porque todas las neuronas están conectadas entre sí.
- No aprovecha la estructura espacial de las imágenes ni el orden de las secuencias.
- Tiende al sobreajuste (overfitting) si no se usa regularización (dropout, L2).

Aplicaciones. Clasificación de clientes, predicción de precios, detección de fraude, diagnóstico a partir de datos clínicos y reconocimiento básico de dígitos.

3. CNN – Red Neuronal Convolucional (Convolutional Neural Network)

Definición. La CNN es una red especializada en datos con estructura de cuadrícula, como las imágenes. En lugar de conectar cada píxel con cada neurona, aplica filtros pequeños que recorren la imagen y detectan patrones locales.

Historia. Se inspira en los estudios de Hubel y Wiesel (1959) sobre la corteza visual. Kunihiko Fukushima propuso el Neocognitron (1980). Yann LeCun desarrolló LeNet-5 (1998) para leer dígitos escritos a mano en cheques bancarios. En 2012, AlexNet (Krizhevsky, Sutskever y Hinton) ganó el concurso ImageNet con amplia ventaja y marcó el inicio del auge del aprendizaje profundo. Después llegaron VGG (2014), GoogLeNet (2014) y ResNet (2015).

Arquitectura.

1. **Capa convolucional:** un filtro (kernel), por ejemplo de 3×3, se desliza sobre la imagen y genera un mapa de características. Las primeras capas detectan bordes; las más profundas, formas y objetos completos.
2. **Función de activación (ReLU):** elimina los valores negativos y añade no linealidad.
3. **Capa de agrupamiento (pooling):** reduce el tamaño del mapa, por ejemplo tomando el valor máximo de cada bloque de 2×2 (max pooling).
4. **Capas totalmente conectadas:** al final, un MLP toma las características extraídas y realiza la clasificación.

$$S(i, j) = \Sigma_m \Sigma_n I(i+m, j+n) \cdot K(m, n)$$

Donde I es la imagen de entrada, K el filtro y S el mapa de características resultante.

Conceptos clave.

- **Pesos compartidos:** el mismo filtro se usa en toda la imagen, lo que reduce mucho los parámetros.
- **Conectividad local:** cada neurona solo ve una región pequeña (campo receptivo).
- **Invarianza a la traslación:** reconoce un objeto aunque cambie de posición.
- **Stride y padding:** el paso del filtro y el relleno de los bordes controlan el tamaño de la salida.

Ventajas.

- Extrae características automáticamente, sin diseñarlas a mano.
- Requiere muchos menos parámetros que un MLP para procesar imágenes.

Limitaciones.

- Necesita grandes cantidades de datos etiquetados y GPU para entrenar.
- No modela bien la dependencia temporal de secuencias largas.
- Es poco interpretable ("caja negra").

Aplicaciones. Reconocimiento facial, vehículos autónomos, diagnóstico por imágenes médicas (radiografías, retinografías, detección de retinopatía diabética), lectura de placas, control de calidad industrial y clasificación de documentos.

4. RNN – Red Neuronal Recurrente (Recurrent Neural Network)

Definición. La RNN es una red diseñada para procesar secuencias, donde el orden importa. Tiene conexiones que regresan sobre sí mismas, lo que le da una especie de memoria: la salida de un paso depende de la entrada actual y de lo que la red procesó antes.

Historia. John Hopfield presentó en 1982 las redes de Hopfield, con conexiones recurrentes. Michael Jordan (1986) y Jeffrey Elman (1990) propusieron las arquitecturas recurrentes simples que se usan como base hasta hoy (la "red de Elman").

Arquitectura. En cada instante de tiempo t, la red recibe una entrada x_t y la combina con su estado oculto anterior h_{t-1} para producir un nuevo estado h_t y una salida y_t. Los mismos pesos se reutilizan en todos los pasos.

$$h_t = tanh(W_hh \cdot h_{t-1} + W_xh \cdot x_t + b_h)$$

$$y_t = W_hy \cdot h_t + b_y$$

Si se "desenrolla" la red en el tiempo, se ve como una cadena de copias de la misma celda, una por cada elemento de la secuencia.

Tipos según entrada y salida.

- **Uno a muchos:** generar una descripción a partir de una imagen.
- **Muchos a uno:** clasificar el sentimiento de una reseña.
- **Muchos a muchos:** traducir una frase de un idioma a otro.

Entrenamiento. Se usa la retropropagación a través del tiempo (Backpropagation Through Time, BPTT): la red se desenrolla y el error se propaga hacia atrás por todos los pasos.

Limitaciones.

- **Desvanecimiento del gradiente:** al multiplicar muchas veces valores menores que 1, el gradiente se vuelve casi cero y la red "olvida" la información de los primeros pasos; por eso le cuesta aprender dependencias de largo plazo (Bengio et al., 1994).
- **Explosión del gradiente:** el caso contrario, cuando los valores crecen sin control; se mitiga con gradient clipping.
- Procesa los datos paso a paso, por lo que no se paraleliza bien y entrena lento.

Aplicaciones. Autocompletado de texto, reconocimiento de voz, predicción de series de tiempo (ventas, demanda, clima), análisis de sentimiento y generación de música.

5. LSTM – Memoria de Largo y Corto Plazo (Long Short-Term Memory)

Definición. La LSTM es un tipo especial de RNN creado para resolver el desvanecimiento del gradiente. Agrega una celda de memoria y un sistema de compuertas (gates) que deciden qué información guardar, qué olvidar y qué entregar como salida.

Historia. La propusieron Sepp Hochreiter y Jürgen Schmidhuber en 1997. Gers, Schmidhuber y Cummins (2000) añadieron la compuerta de olvido, que hoy forma parte del diseño estándar. Entre 2014 y 2017 fue la base de los sistemas de traducción automática y reconocimiento de voz de grandes empresas. En 2014, Cho et al. propusieron la GRU, una variante simplificada con dos compuertas.

Arquitectura. Cada celda LSTM mantiene dos estados: el estado de la celda C_t, que funciona como una "cinta transportadora" de memoria de largo plazo, y el estado oculto h_t, que es la memoria de corto plazo y la salida. Las compuertas usan la función sigmoide (σ), que da valores entre 0 (bloquear) y 1 (dejar pasar).

1. **Compuerta de olvido (f):** decide qué parte de la memoria anterior se descarta.
2. **Compuerta de entrada (i):** decide qué información nueva se guarda, junto con un candidato $\tilde{C}$ calculado con tanh.
3. **Actualización de la celda:** combina lo que se conserva con lo nuevo.
4. **Compuerta de salida (o):** decide qué parte de la memoria se entrega como h_t.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot tanh(C_t)$$

Cómo resuelve el problema de la RNN. El estado de la celda se actualiza principalmente con sumas, no con multiplicaciones repetidas. Así el gradiente puede fluir por muchos pasos sin desaparecer, y la red recuerda información de cientos de pasos atrás.

Ventajas.

- Aprende dependencias de largo plazo en secuencias.
- Es más estable de entrenar que una RNN simple.

Limitaciones.

- Tiene unas cuatro veces más parámetros que una RNN simple, por las compuertas.
- Sigue procesando en secuencia, por lo que entrena más lento que un Transformer.
- En secuencias muy largas ha sido superada por los mecanismos de atención.

Aplicaciones. Traducción automática, reconocimiento de voz, predicción de series financieras y de demanda, generación de texto, subtitulado automático de video y detección de anomalías en sensores.

6. Cuadro comparativo

Criterio	MLP	CNN	RNN	LSTM
Tipo de dato	Tabular, vectores	Imágenes, video, datos en cuadrícula	Secuencias cortas	Secuencias largas
Flujo de información	Solo hacia adelante	Solo hacia adelante	Con ciclos (recurrente)	Con ciclos + celda de memoria
Componente clave	Capas densas	Filtros convolucionales y pooling	Estado oculto	Compuertas de olvido, entrada y salida
Memoria	No tiene	No tiene	Corto plazo	Largo y corto plazo
Entrenamiento	Backpropagation	Backpropagation	BPTT	BPTT
Parámetros	Muchos (todo conectado)	Pocos (pesos compartidos)	Pocos (pesos reutilizados en el tiempo)	Unas 4 veces los de una RNN
Principal limitación	No capta estructura espacial ni temporal	Requiere muchos datos y GPU	Desvanecimiento del gradiente	Costo computacional; entrena en serie
Hito histórico	Rumelhart et al., 1986	LeNet-5, 1998; AlexNet, 2012	Elman, 1990	Hochreiter y Schmidhuber, 1997
Aplicación típica	Predicción de precios, fraude	Visión artificial, imágenes médicas	Autocompletado, series de tiempo	Traducción, reconocimiento de voz

7. Conclusiones

1. Cada arquitectura responde a un tipo de dato: el MLP es la base general, la CNN domina el procesamiento de imágenes, y la RNN y la LSTM se especializan en secuencias.
2. Las arquitecturas evolucionaron para superar las limitaciones de la anterior: la CNN reduce los parámetros del MLP en imágenes, y la LSTM resuelve el desvanecimiento del gradiente de la RNN.
3. Los modelos modernos combinan varias de ellas; por ejemplo, una CNN extrae características de una imagen y una LSTM genera su descripción en texto.
4. Tendencia actual: desde 2017, la arquitectura Transformer (Vaswani et al.) reemplazó en gran parte a las RNN y LSTM en procesamiento de lenguaje, porque su mecanismo de atención procesa toda la secuencia en paralelo. Es la base de los modelos de lenguaje grandes actuales. En visión, los Vision Transformers (ViT) compiten con las CNN.

5. Aun así, las CNN siguen siendo muy usadas en dispositivos con pocos recursos, y las LSTM continúan vigentes en series de tiempo y sistemas embebidos, donde son más ligeras que un Transformer.

8. Referencias

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Transactions on Neural Networks, 5(2), 157–166.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. Proceedings of EMNLP 2014, 1724–1734.

Elman, J. L. (1990). Finding structure in time. Cognitive Science, 14(2), 179–211.

Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. Neural Computation, 12(10), 2451–2471.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. https://www.deeplearningbook.org

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems, 25.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.